UNIT 3: RELATIONAL DATABASE DESIGN & NORMALIZATION

3.1 Purpose of Database Normalization

Normalization is a systematic technique of designing relational schemas to minimize data redundancy and eliminate update, insertion, and deletion anomalies. By decomposing tables with excessive dependencies into smaller, well-structured relations, normalization ensures that every non-key attribute directly depends on candidate keys.

3.2 First Normal Form (1NF)

A relation is in First Normal Form if every attribute contains only atomic (indivisible) values, and there are no repeating groups or multivalued columns within any tuple. Every row must be uniquely identifiable by a primary key.

3.3 Second Normal Form (2NF)

A relation is in Second Normal Form if it is in 1NF and contains no partial functional dependencies. That is, every non-prime attribute must be fully functionally dependent on the entire primary key, rather than on a proper subset of a composite candidate key. If the primary key is a single attribute, 1NF relations automatically satisfy 2NF.

3.4 Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF)

3.4.1 Third Normal Form (3NF)

A schema is in 3NF if it is in 2NF and there is no transitive dependency. Formally, for every non-trivial functional dependency X -> Y, either X is a superkey, or Y is a prime attribute. In simple terms, non-key attributes must depend on nothing except the primary key.

3.4.2 Boyce-Codd Normal Form (BCNF)

BCNF is a stricter form of 3NF. A relation is in BCNF if for every functional dependency X -> Y, X must be a superkey. BCNF resolves anomalies arising when a table has overlapping candidate keys.

3.5 Indexing Techniques in Relational Databases

Database indexing optimizes data retrieval speed without scanning every disk block:

1. B-Tree and B+ Tree Indexing: Balanced search trees that keep data sorted and allow search, sequential access, insertions, and deletions in logarithmic time O(log n). B+ trees store all records in leaf nodes linked sequentially, making range queries exceptionally efficient.
2. Hash Indexing: Uses an internal hash function to map search keys to specific disk buckets, offering O(1) average lookup time for point queries (exact equality matches), but unable to support range comparisons.